

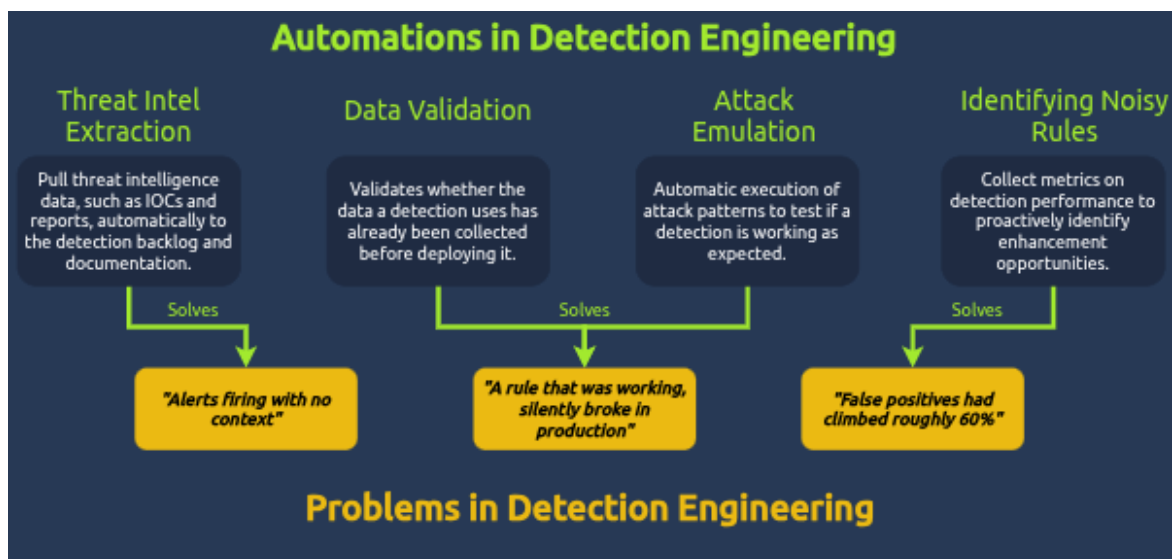
L2 - 7.5. AI & Automation in Detection Engineering

Task 1 - Introduction

This room is about how detection engineers solve that with automation and AI. Automation provides a structured environment for detection engineering, and AI accelerates the work within that structure.

Task 2 - Automation & Detection-as-Code

In this task, you will explore how detection engineers are using automation to deliver at a good pace while keeping the detection quality, and help the detection engineers at Aurora Logistics.

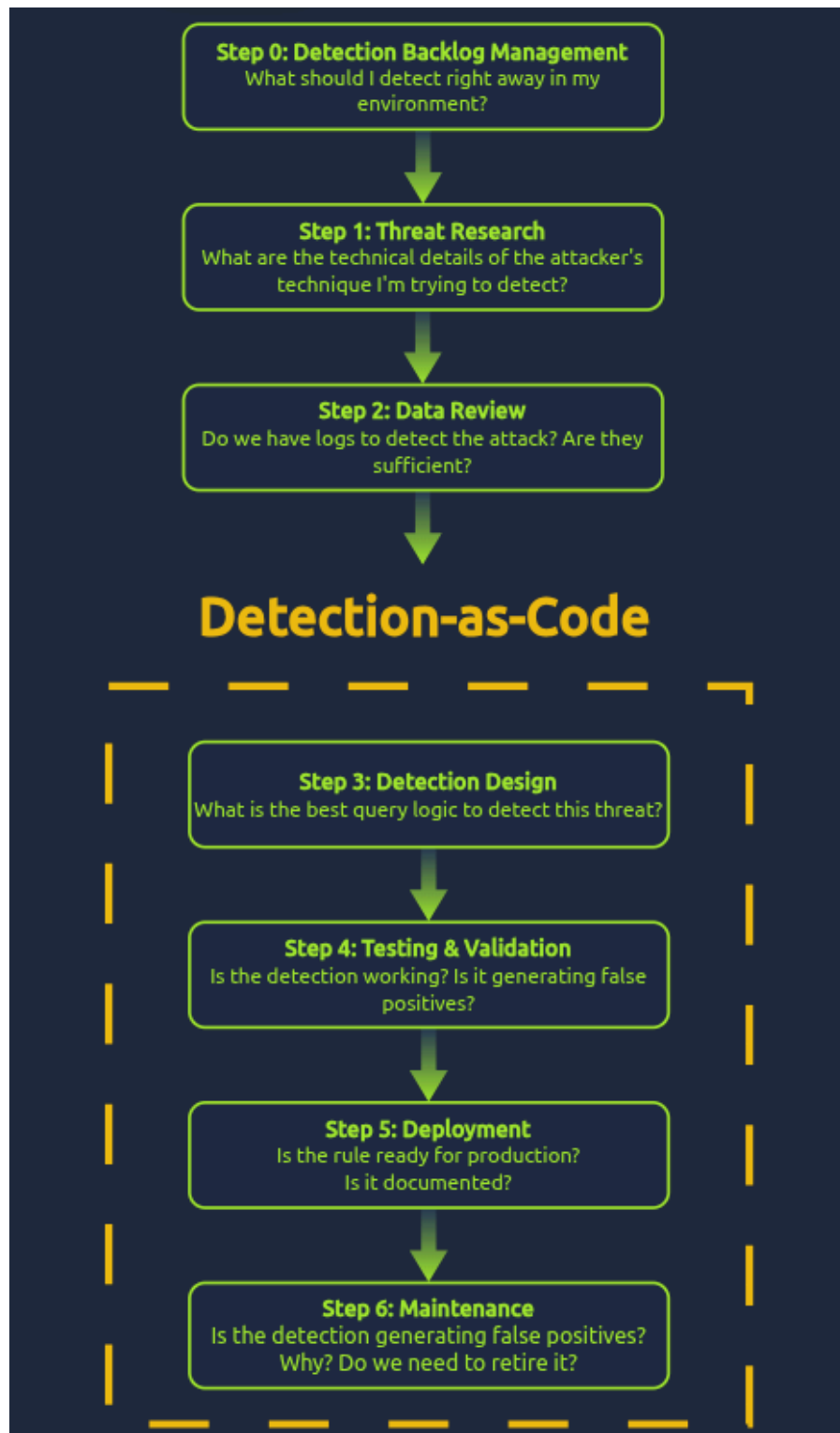


There is one stretch of the lifecycle that detection engineers actually build and own as a repeatable machine. The stretch from designing a rule to running it in production. That "machine" is called **Detection-as-Code (DaC)**.

With DaC, detection engineers treat a detection, usually a Sigma rule YAML file, as a file in a Git repository (like a code file), and apply the engineering strategies:

- **Version control (Git):** Every change to every rule is tracked: who changed what, when, and why. You can compare versions and roll back instantly when something breaks. The repository, not the SIEM, becomes the single source of truth for your detections.
- **Peer review (pull requests):** A pull request (PR) is a proposed change that a second engineer must approve before it is merged. Review stops being a favor you ask a colleague and becomes a normal, low-friction step.
- **Automated checks (a CI/CD pipeline):** CI/CD (Continuous Integration/Continuous Deployment) is an automated assembly line that runs whenever you propose a change. For a detection, that line checks the rule's syntax, enforces your team's standards, and tests the rule against live data to estimate how noisy it will be. For example, that 60% jump in false positives would have been flagged by the pipeline at proposal time.
- **Controlled deployment:** Once approved and merged, the *same* pipeline automatically pushes the rule into the SIEM, consistently every time. No more hand-deploying and hoping you did it right.

Where DaC Fits in the Lifecycle



- **Backlog Management (Step 0)** is typically tracked in a project or issue system such as Jira, where each detection requirement and its priority are represented as a ticket.

- **Threat Research (Step 1)** can be accelerated by automatically consuming threat intelligence reports, feeds, and TI platforms that parse what comes in and surface the techniques and indicators worth detecting, rather than having someone read every report by hand.
- **Data Review (Step 2)** is far quicker when an automated inventory tells you what data actually lives in your SIEM, so you can answer "do we even collect the logs to catch this?" before you write a line of detection logic.

Why DaC Lives Outside the SIEM

Detection as Code must live outside the SIEM to provide centralized version control, automated testing, and unified governance across multiple detection tools that native SIEMs lack.

Answer the questions below

What type of change request lets a second engineer review a detection before it reaches production?

pull request

What file format are Sigma rules stored in?

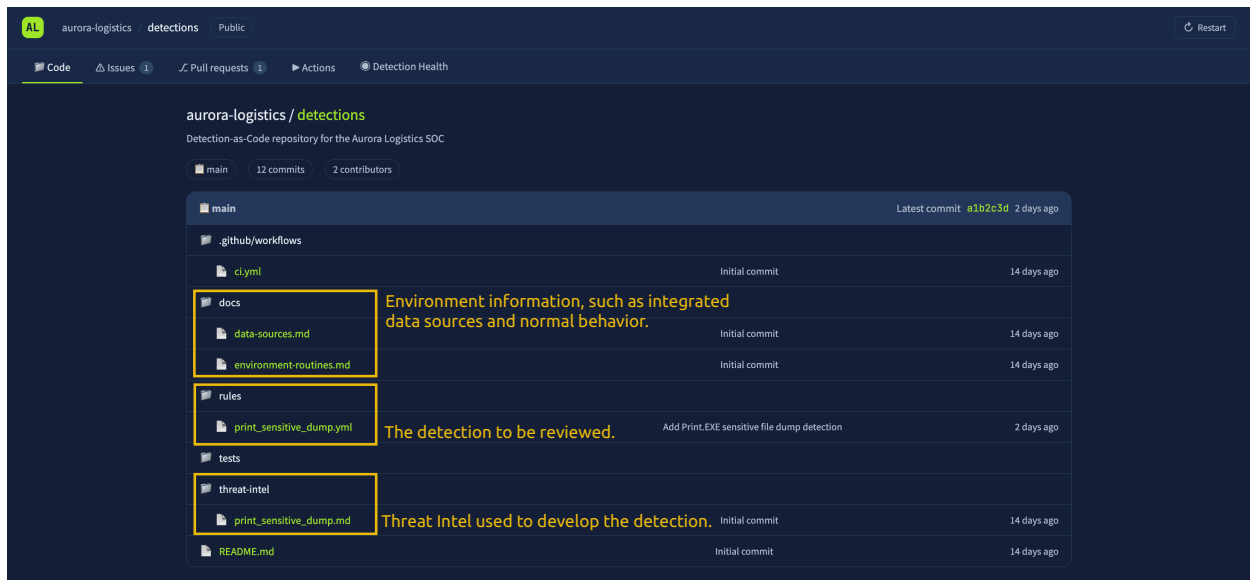
YAML

What is the most common type of company that heavily uses Detection-as-Code?

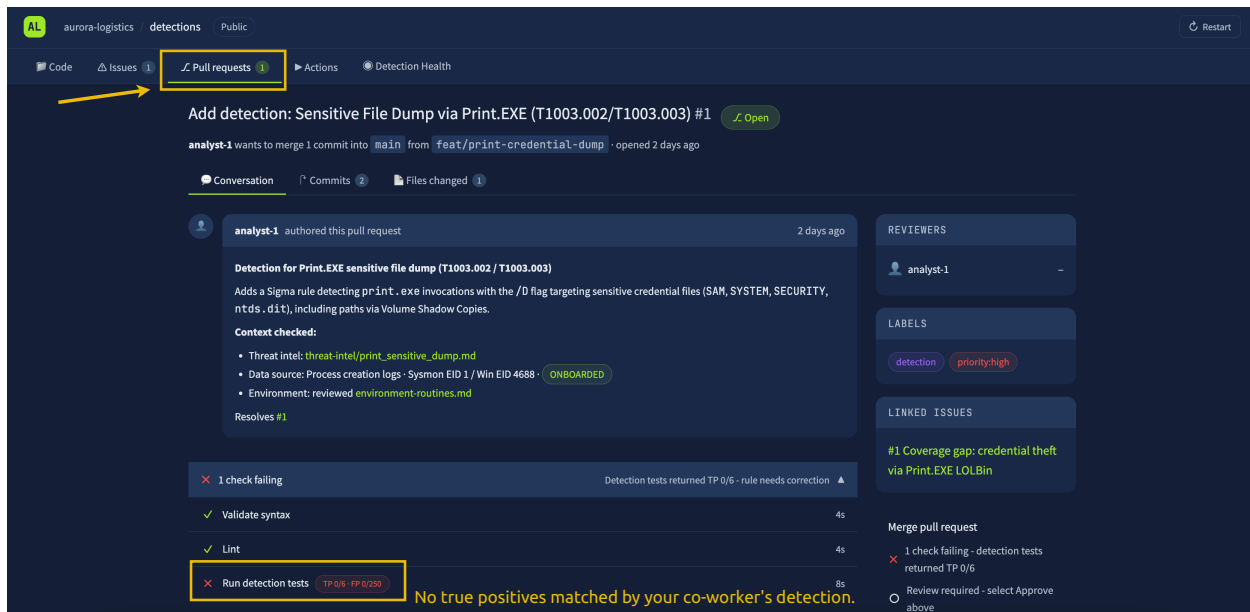
MSSP SOC

Task 3 - Detection-as-Code: Reviewing a Detection

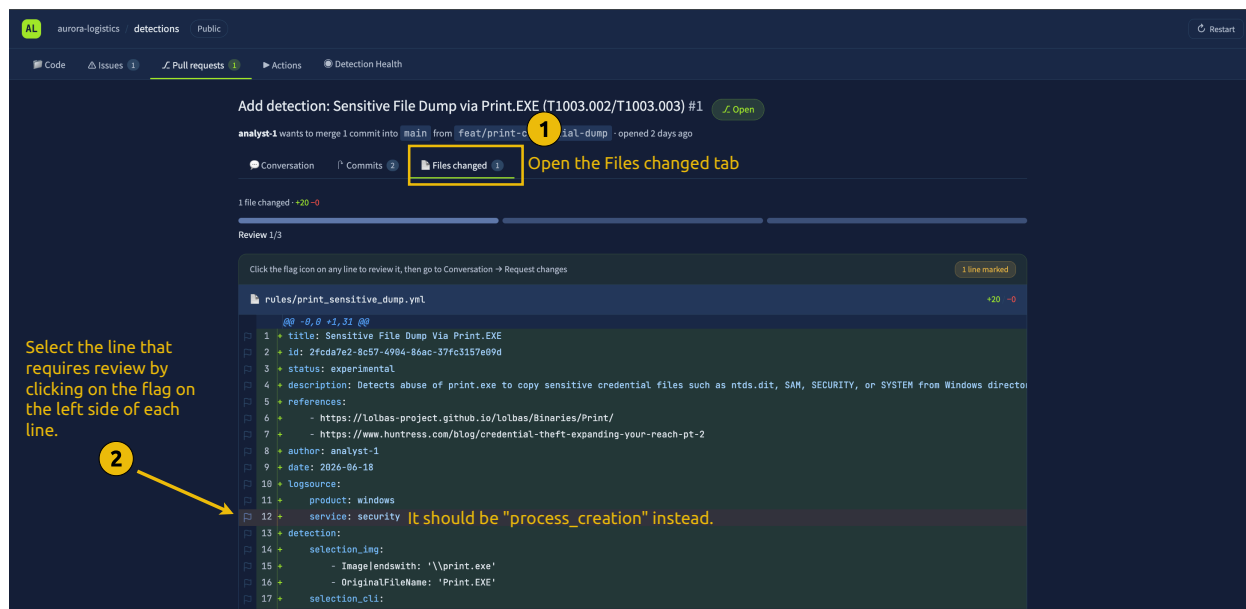
- **The detection idea (backlog):** the Aurora team recently identified a priority to detect different LOLBins used for credential theft. In this case, the use of `print.exe` to dump `ntds.dit`.
- **The research (the technical background).** A linked threat-intel document explains the technique, and it was used to build the detection. It can be found under the threat-intel folder inside the repository.



Stage 1: Review the Detection (Pull Request): Review the open pull request for the new `print.exe` detection under the Pull Requests tab, checking the conversation and pipeline run results before approving your teammate's change.



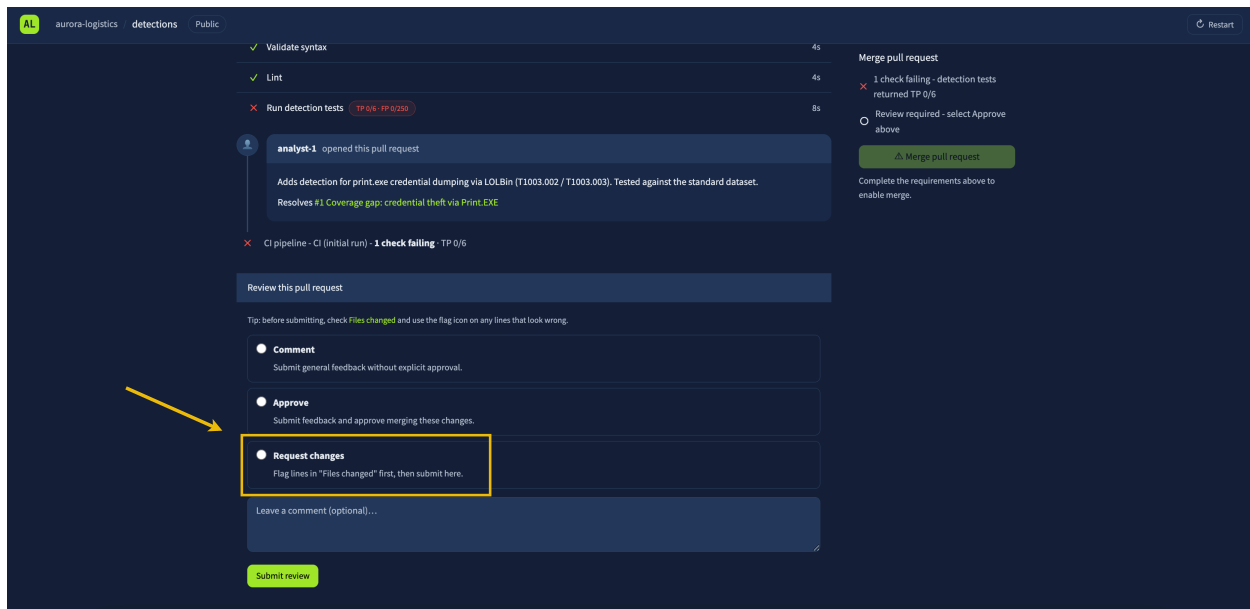
Review the `Files Changed` tab to fix the logic: change the invalid data source to `process_creation`.



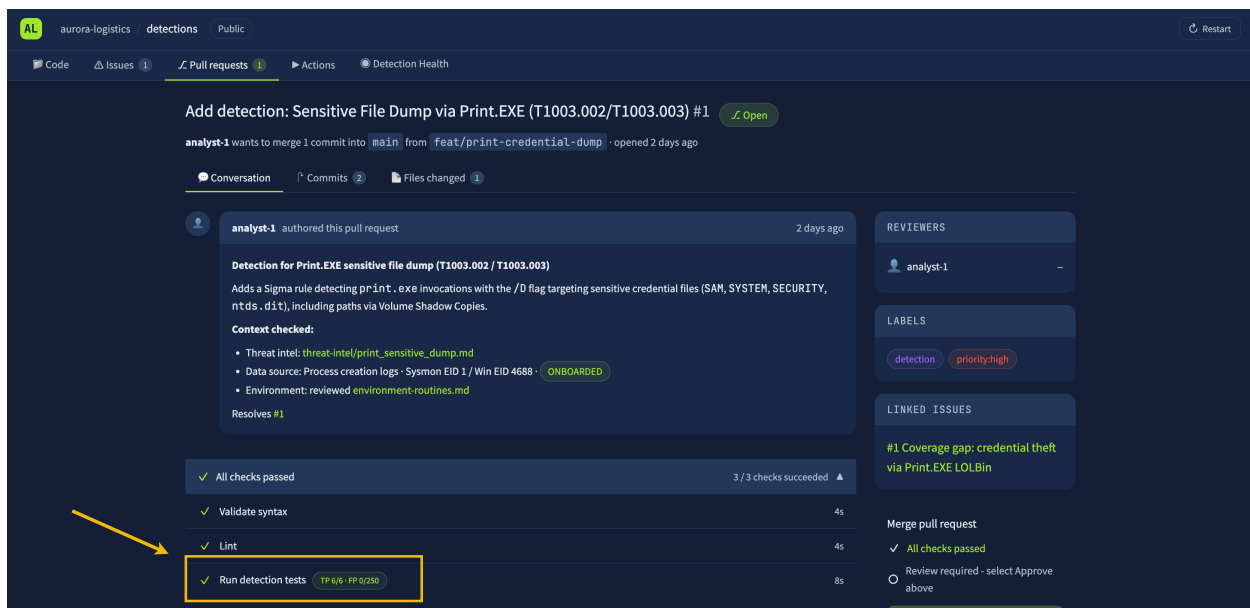
There are **two other** problems with that detection. Walk the review deliberately, using the intel as your checklist:

- **Does the logic match the intel?** Check whether the process and file names match the threat intel data.
- **Is the metadata complete?** Description, references, author, severity, ATT&CK tags. Are they all present? In DaC, missing or sloppy metadata is a valid reason to request a change.
- **Is there a Sigma mistake?** Even with a syntax test, the rule can be semantically perfect but still contain logical mistakes. Is anything missing to be expressed in the condition?

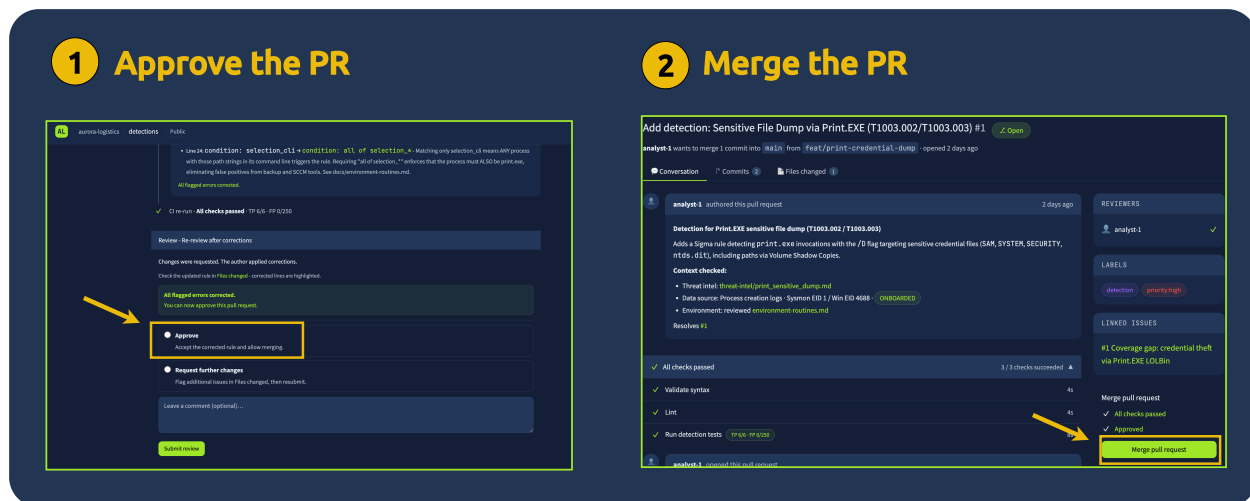
Stage 2: Requesting Changes: Select **Request changes** in the Conversation tab and submit a comment detailing the logic errors and required updates.



Once all three corrections are made, the re-run CI pipeline successfully matches all true positives.



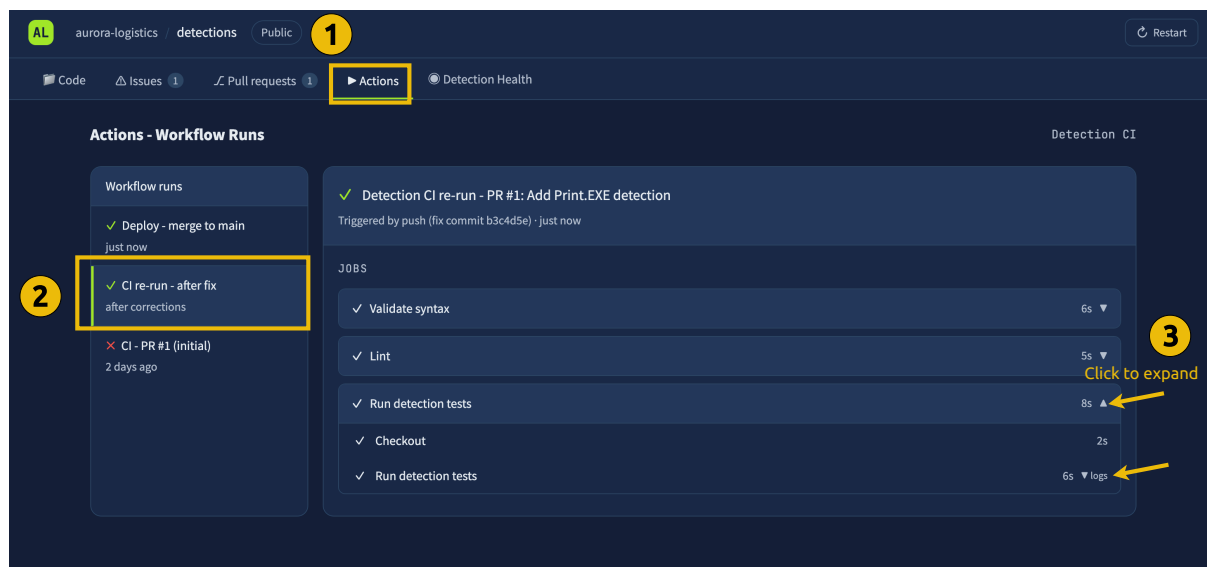
Stage 3: Deploying the Detection: Approve and merge the clean pull request to trigger the deployment pipeline, converting the Sigma rule into SIEM queries and pushing it live via API.



Open the **Actions** tab to see the full pipeline history for this pull request. Three runs are listed:

- The initial CI run that failed when the detection was first submitted.
- The re-run triggered automatically after your corrections were applied.
- The deploy job that fired the moment the pull request was merged.

Click into the re-run pipeline execution and expand the **Run detection tests** step: scroll through the verbose output carefully. *The pipeline left something for you.* That full record is one of the core guarantees of Detection-as-Code: every run is timestamped and attributed, so you can always trace which rule version was live, who approved it, and what the tests said at the time.



Answer the questions below

What is the flag found after successfully reviewing the detection?

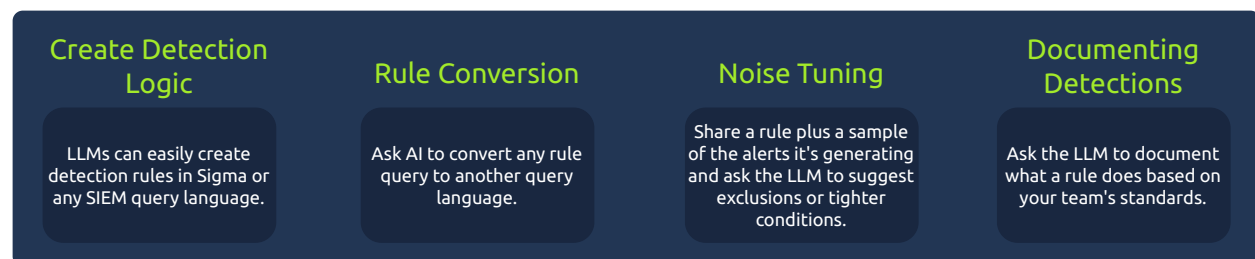
THM{pr0c_cr34t10n_I0gs0urc3_fix3d}

Which detection should be prioritized for a tuning activity?

Suspicious PowerShell Encoded Cmd

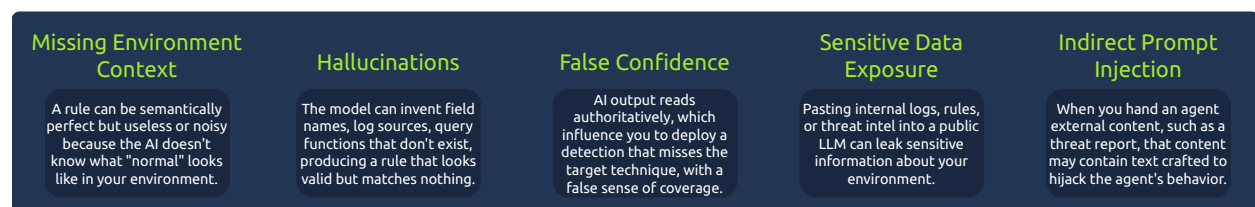
Task 4 - AI in Detection Engineering

Using AI in Detection Engineering: LLMs are powerful for boosting detection engineering work. Even used standalone, as nothing more than a co-pilot, they can take a meaningful chunk of repetitive effort off an engineer's plate:



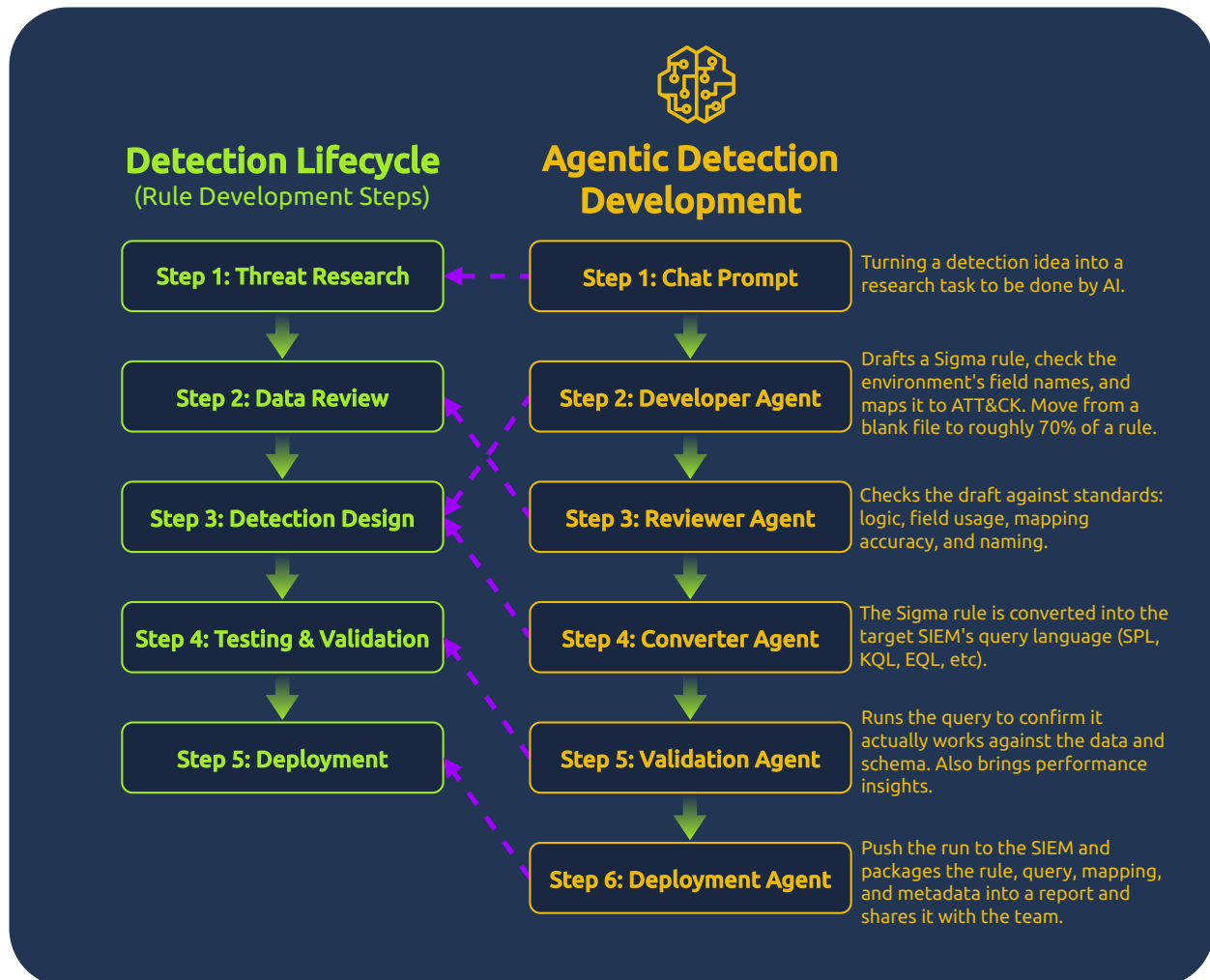
While AI provides the speed needed for modern detection engineering, human judgment remains essential to design thoughtful, context-aware detections and ensure safety.

Risks of AI in Detection Engineering: You're probably tired of hearing that AI can make mistakes, and detection engineering is no exception. Relying on AI here carries some specific risks worth keeping in mind:



All risks share a theme: the judgment about what to trust. As a detection engineer, you should take a step back from the AI velocity and stay alert to the gaps and mistakes it can introduce.

Agentic Detection Development: AI accelerates detection engineering by chaining multiple LLM agents into a unified pipeline that automates the lifecycle from plain-language requests to tested detections.



Rule creation is not the only place AI earns. It also works the **bookends** that this pipeline does not touch:

- At the front, it can read an incident-response write-up or a threat-intel report, extract the indicators and techniques worth detecting, and feed requirements into **Threat Research (Step 1)** far faster than reading line by line.
- At the back, pointed at your alert history, it can flag which live detections are noisy and worth tuning or retiring, the **Maintenance (Step 6)** stage.

Note that the agentic pipeline, including its own "reviewer agent," does not remove the human review step from your Detection-as-Code pipeline. The AI reviewer is a fast quality pre-check, not a person signing off. What it really buys you is speed: AI hands you a drafted, converted, tested, and documented detection in minutes. That output still enters the same DaC pipeline you used in Task 3, the human pull-request review, the CI tests, and the controlled deploy before it touches production.

Answer the questions below

What is the name of the risk related to when a person feeds AI with external documentation that has instructions to hijack the agent's behavior?

Indirect Prompt Injection

What is the name of the risk related to when the AI provides a technically wrong answer, but with a feeling that it's 100% correct?

False Confidence

Which AI agent on an Agentic Detection Development pipeline is responsible for validating if the detection is working and not generating false positives?

Validation Agent

Task 5 - Practice: Detection Engineer Copilot

Scenario: The DCSync Abuse: You are still working at Aurora Logistics, which runs a hybrid Active Directory/Microsoft Entra ID (Azure AD) environment, with identities synchronized between them via Azure AD Connect. Because credentials are stored and managed in the on-premises Active Directory, your domain controllers are a high-value target: anyone who can impersonate a DC and pull credential material can compromise the entire identity backbone of both environments. This is exactly what a DCSync attack does. By abusing the replication privileges, an attacker can request that a real DC replicate password hashes for any account, including krbtgt and privileged admins, without ever running code on the DC itself. Based on this, your manager asked you to build a detection for it. Rather than writing the rule from scratch, you decide to put the

team's new AI workflow to the test and have it draft the detection for you, then apply your own detection engineering judgment to find what it gets wrong.

Step 1: Prompting for the Detection : Open the AI Agent attached to this task by clicking the `Open Agent` button. Then request the creation of a DCSync attack Sigma rule. You can use a simple prompt such as the one below:

"Create a Sigma rule to detect DCSync attacks in an Active Directory environment."

Now, validate if the generated detection matches the technical details of a DCSync attack:

- Check if it's using Event ID 4662, which is the core event that captures directory replication requests.
- Check the replication property GUIDs in the `Properties` field. A DCSync detection must look for the control access rights `1131f6aa-9c07-11d1-f79f-00c04fc2dcd2` (`DS-Replication-Get-Changes`) and `1131f6ad-9c07-11d1-f79f-00c04fc2dcd2` (`DS-Replication-Get-Changes-All`).
- Check whether it filters out legitimate replication sources. Real domain controllers perform this operation continuously as part of normal business operations.
- Check whether the rule respects the Sigma syntax. Are all conditions being used? Is there any bad usage of the Sigma language?

Step 2: Validating the Detection: One of the abilities of this Detection AI-copilot is to test a detection rule against environment data. After reviewing the sigma rule, you can ask it to run the detection against the company data:

"Test the generated detection against environment data and generate a summary of the matched events to help me identify potential rule gaps."

As you can see, the agent reported that a known service within Aurora's environment triggered some alerts. Here's the trap: the rule itself is semantically perfect with valid syntax, good logic, the correct event, and replication operations.

As an exam answer, it scores full marks. But the AI Agent lacked context about Aurora's internal environment, which explains the alerts it generated in the first version. The model knows detection engineering in general, but nothing about your environment in particular. A detection is only meaningful against a baseline of normal activity, and the AI has never seen yours, so it flagged the internal benign behavior as an attack.

Step 3: Adjusting the Detection: Now that you've spotted the missing context of your environment, you can ask the AI agent to filter out this specific behavior and tune the detection. After successfully tuning the detection, ask the AI agent to test the detection and grab the flag in the response to complete the room!

Answer the questions below

| What is the flag generated by the AI agent?

THM{A1_D3T3CT1ON_ENG}

Task 6 - Conclusion

Detection engineering at scale comes down to a simple division of labor: automation provides the structure to ship detections safely and repeatably, and AI accelerates the slow, human-intensive work within that structure. Throughout this room, you saw how Detection-as-Code turns rule development into a reviewable, testable, version-controlled pipeline, and how AI agents can draft, convert, and test a detection in minutes. But you also saw the catch: speed without judgment is dangerous, and the detection engineer's review remains the safeguard that keeps fast detections trustworthy.